

Building the Ethereum Universal Signer

DIY or pre-built: architecture and trade-offs

Nicolas Bacca (btchip)
Chef, zknox


Part 1
WHY NOW?
ABUNDANCE_REGARDLESS_OF_MEANS

Hardware wallet incidents

COLDCARD

Entropy
Introduced Mar 2021
Disclosed Jul 2026

Lack of audits. Entropy is hard to audit, and it is a critical dependency.

LEDGER

Implant
Introduced 2025 (est.)
Disclosed Oct 2026

Scope of genuineness and control of the supply chain.

KEYSTONE

Genuineness key leak
Introduced 2023
Disclosed Oct 2026

The genuineness implementation is part of the security.

AI CHANGES EVERYTHING

Agents write firmware, (almost) route PCBs and design cases, and flash devices.

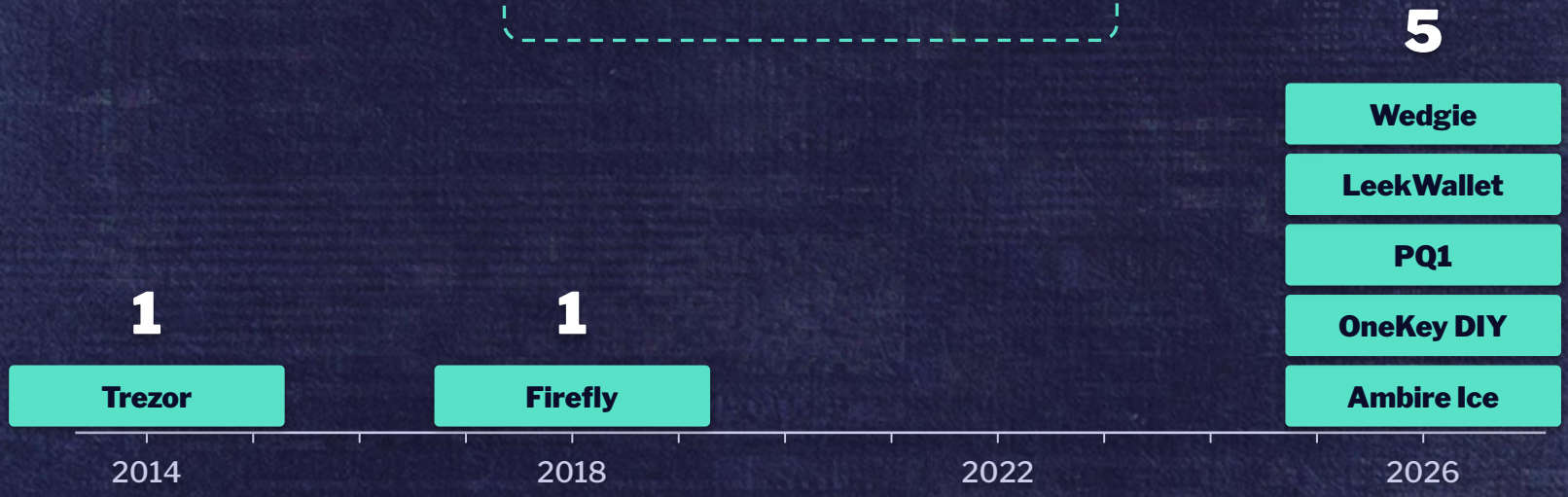
Building your own hardware wallet is no longer a niche hobby.

Everybody can be a hardware wallet vendor now.



DIY HARDWARE WALLETS: 2026 IS THE YEAR

+ any device, generically
ESP-Mosaico (mosaico.ai, Espressif, Oct 2026): modular hardware where agents write and flash the firmware



DIY / open builds by first availability year. Trezor: open hardware since launch; Firefly: hackaday.io project.

Part 2

THE ETHEREUM UNIVERSAL SIGNER



COMPONENTS OF A HARDWARE WALLET

Seed management

Always needed, at least as a generator for other secrets. Stateful or stateless.

Cryptography

Curves, hashes, BIP-32/39 derivation, entropy

Genuineness

Prove the device is what it claims to be

Ethereum signing

Decode, display, sign: transactions, EIP-712, EIP-191

Application management

Load, isolate, update applications

UX

Screens and inputs: what the user actually verifies

Transport

QR (ERC-4527), USB, BLE

Each component can be tackled separately, by different teams or agents.

OPINIONATED ETHEREUM SIGNING

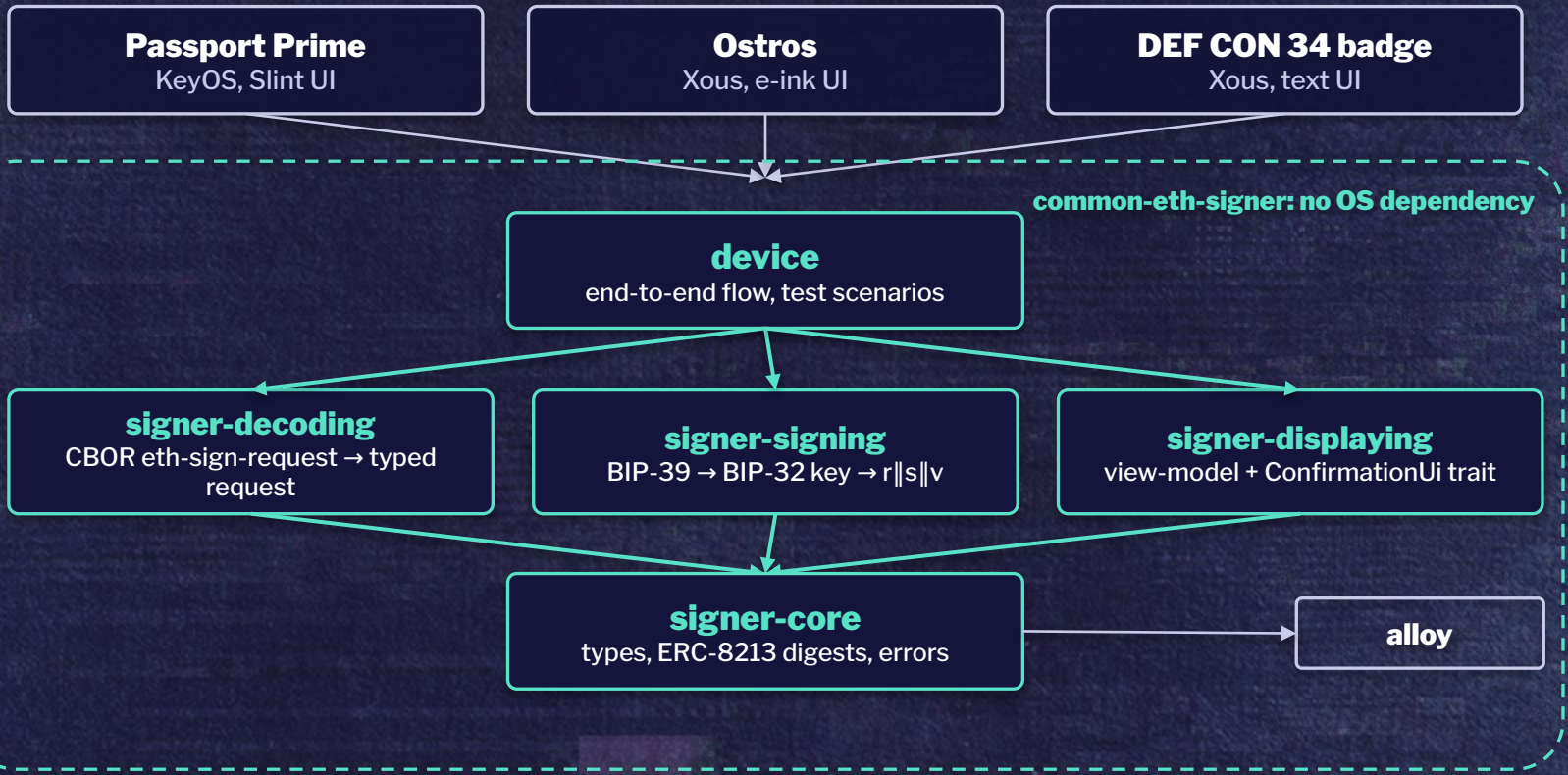
- **Rust**: more guarantees at compile time, ideal for agents, built on alloy
- **Airgap-ready**: ERC-4527 QR requests and signatures
- **Minimalistic**, while promoting modern standards: pick only the modules you need
- **CROPS in mind**: verifiable signing (ERC-8213) before clear signing (ERC-7730)

PLEASE STOP YAPPING



https://github.com/ethereum/ethereum-app

INSIDE COMMON-ETH-SIGNER



EXISTING IMPLEMENTATIONS

Passport Prime
Foundation

- Commercial
- KeyOS multi-app OS
- Slint UI
- Camera (QR)
- Later: USB, BLE

Ostros
Xous · ostros profile

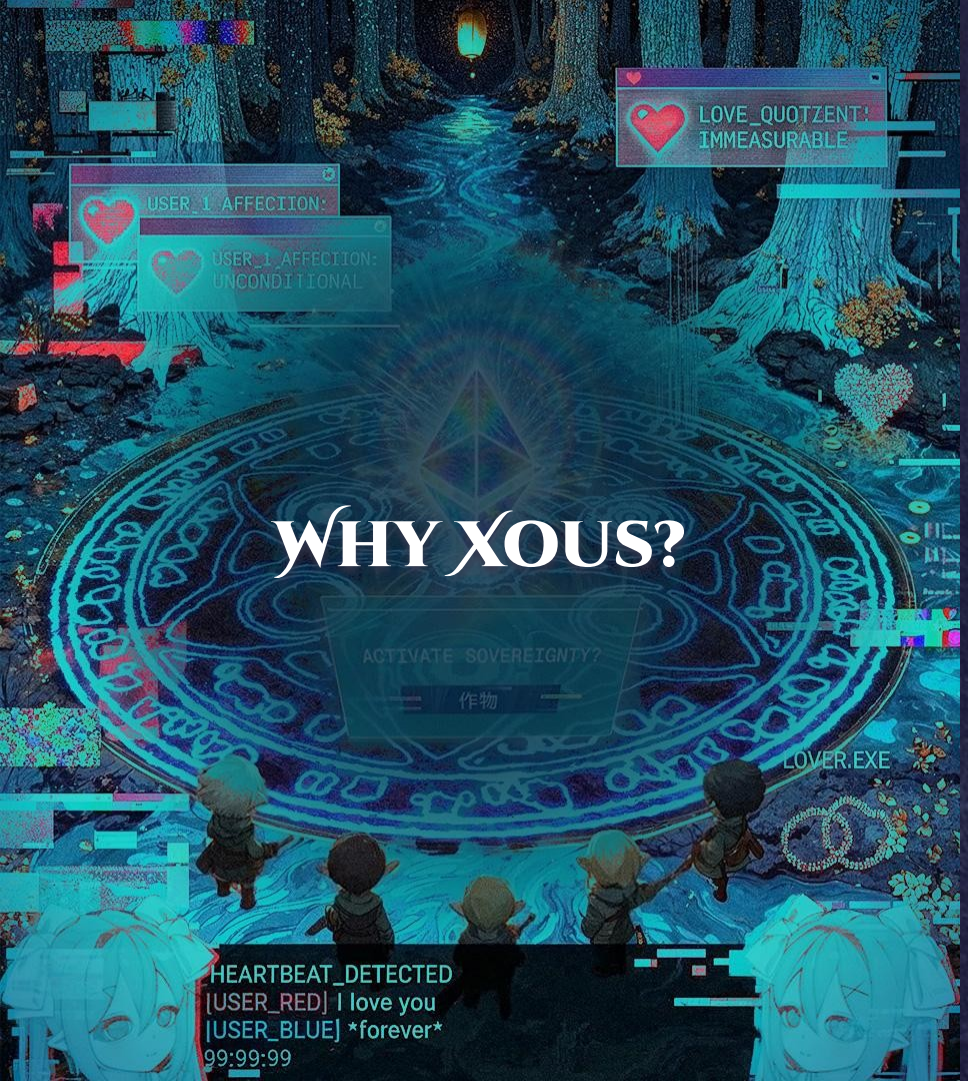
- Commercial
- Xous
- e-ink touch UI
- USB HID
- Later: BLE

DEF CON 34 badge
Xous · baosec-lite

- Text UI, jog dial
- Custom-focus camera (QR)

Dabao
Xous · dabao profile

- No UI
- Bare signer for developers



- **Microkernel** targeting RISC-V with an MMU
- **Open implementation**: VexRiscv-based, RTL available
- **Verifiable** from code to chip, including secure boot
- **Designed to implement Rust memory management in hardware** (mutable XOR shared)

Adding an e-ink UI to Xous

eth-signer app
declares screens + Intent, never a waveform

↓

einkui-core / einkui-xous
Level A: postcard widget tree over Xous IPC

↓

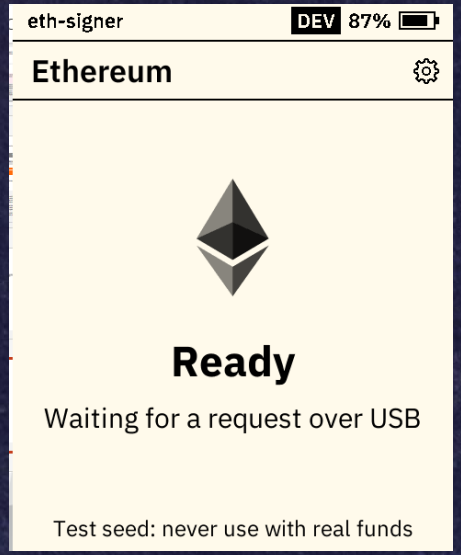
einkui renderer service
layout, fonts, touch, hit-testing

↓

einkui-waveform planner
Intent → INIT / DU / GC16 / GL16 / A2

↓

ED028TC1 panel | einkui-panelsim
real panel, or host simulation of flashing and ghosting





Ostros: idle screen and “slide to sign” with the ERC-8213 calldata digest

What's next for the Universal Signer

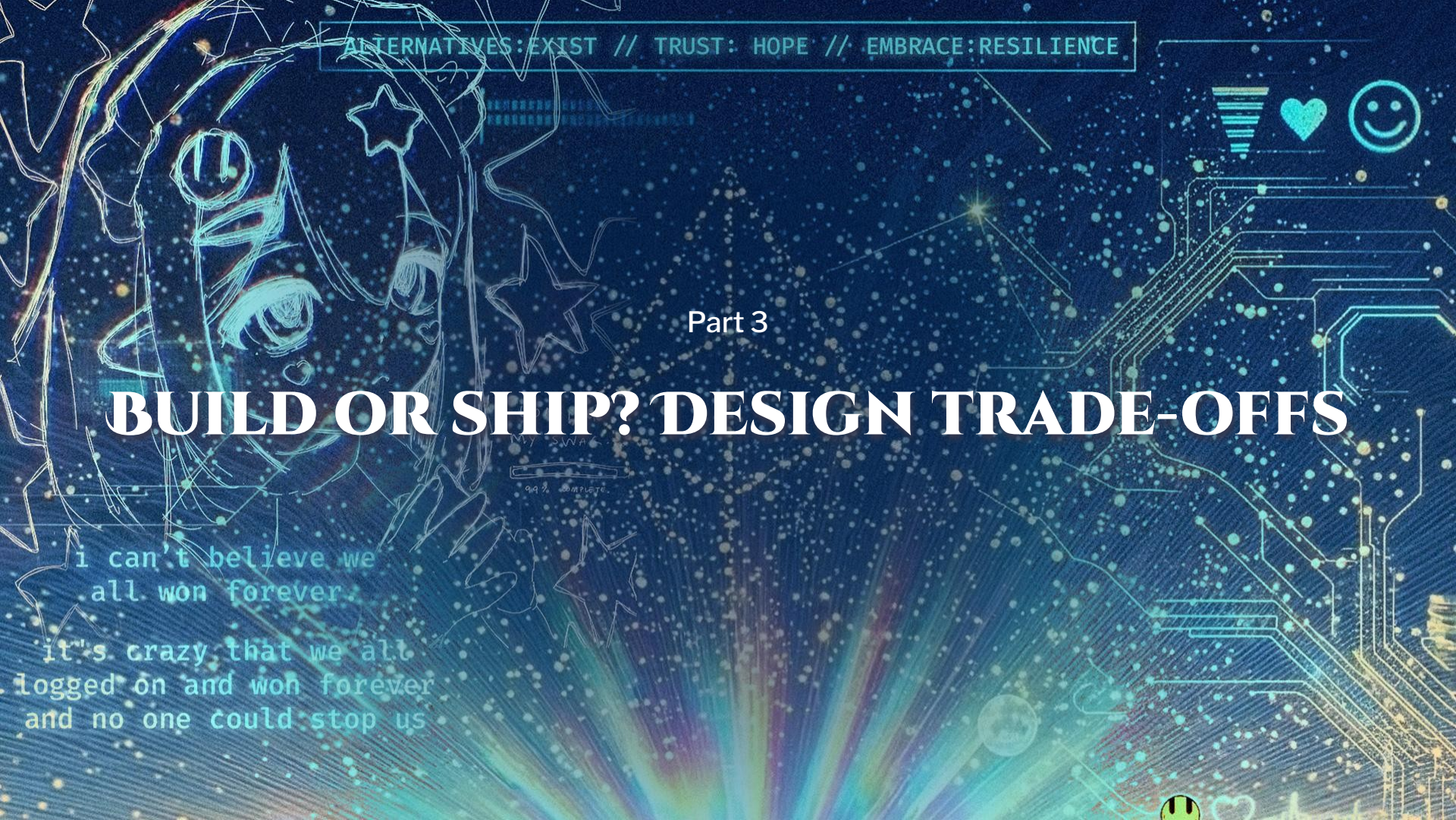
Portable clear signing

- ERC-7730 descriptors, interpreted the same way on every device
- Today each hardware wallet has its own internal format

More implementations

- New devices, OSes and transports. Lean code yours!





Part 3

BUILD OR SHIP? DESIGN TRADE-OFFS



DIY KIT OR SHIP A DEVICE?

DIY kit: point to Bill Of Materials, no added trust or vendor genuineness claims, no direct profit

Shipped device: hopefully profitable, added trust, possible vendor genuineness claims

How do you verify what's running on the device?

- Dumping the device memory from a device application is dangerous: compromised code can lie
- You can be creative (PQ1), but be careful if depending on security features of a non-hardened chip (such as locked memory areas)
- Audit once, or all the time (evil maid)?
- Chip debug features: are they available, and usable by the user? Don't leave them on afterwards
- ROM doesn't really exist nowadays

Multiple applications on a hardware wallet

Open source firmware

No brainer, the most common

- Flash whatever firmware you like
- One app = the whole device

Multi-application OS

Ledger, Passport Prime

- Needs a loader + protection against malicious apps
- Usually: each app locked to a BIP-32 path
- Passport Prime: a new seed per app, no central vetting, but switching wallets is harder

Xous model

Loader as an application

- The loader can be an application itself
- Process server can check metadata permissions

STATEFUL OR STATELESS?

Stateful

- Must be hardened against theft
- More chips, usually proprietary secure elements (STSAFE, Optiga, ATECC) - or tricks (Trezor T SD card)
- Some open exceptions: TROPIC01, Daric

Stateless

- Storage doesn't need to be hardened - harden the executed code path
- Seed from an external resource (SD card on a Raspberry Pi, SeedQR + passphrase)
- Still has to trust the bootloader



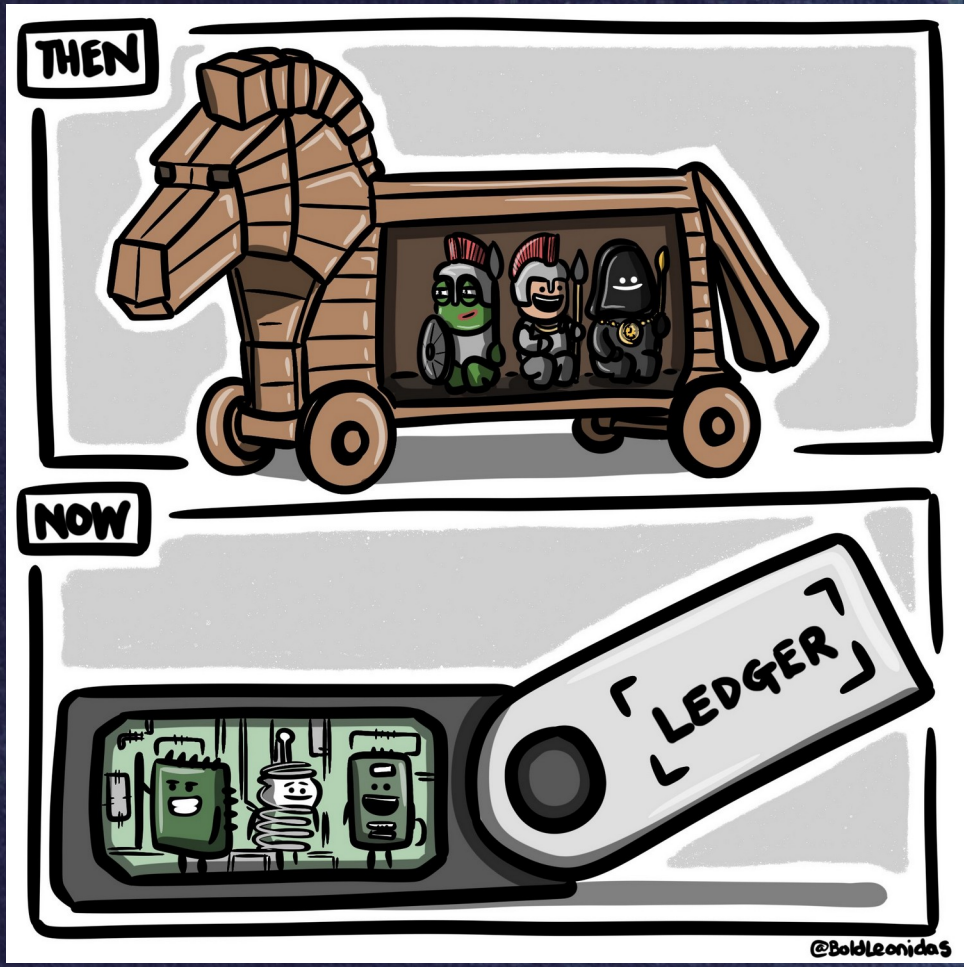
GENUINE CHECK

- Not only for security: it can also be a commercial hedge
- Needs a strongly protected secret on the device
- One key per device: privacy issue. Shared key: dangerous, depending on batch size
- Scope must be explicit: at least the chips running the code + the chips holding the secrets

Implants: The Ledger Case

The secure element is untouched.

The implant taps the SPI bus of the screen, reads what is displayed, and exfiltrates it over a cellular modem.

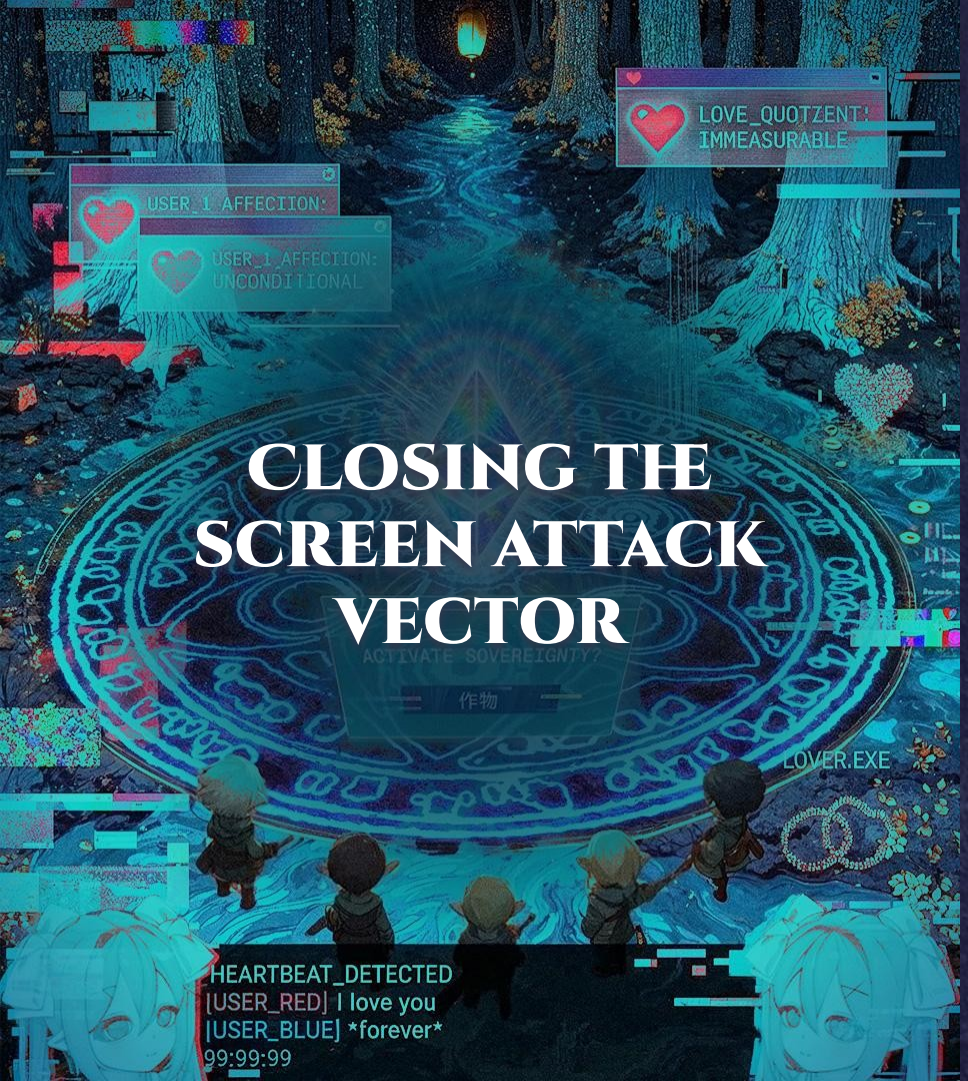


IMPLANTS

- Think of implants like modchips: they exploit complexity
- **Detect**: make the device fully observable (tear down + reassemble). A transparent case is not enough, typically if the implant is hidden or integrated
- **Passively harder**: hard to open (ultrasonic welding, not glue), potting + transparent. Hurts observability, wires still reachable
- **Actively harder**: mesh + battery, erase on tamper. Hurts observability
- **Implant judo**: detect its specific characteristics in the field (Mark Karpeles)

- **Single-chip** solution, or cryptographically **paired chips**
- Win if the attacker needs **both an implant and malware** to do harm
- Remaining attack vector: the **screen**, when generating or restoring the mnemonic, or entering a passphrase





CLOSING THE SCREEN ATTACK VECTOR

- Needs genuineness
- **24-word seed**: 12 words on the device screen, 12 words exchanged securely with the computer (verify that the session key was generated by the device)
- **Alternative**: 12-word seed + a word permutation exchanged securely with the computer
- An implant on the screen alone never sees the full secret

My ideal implementations

DIY

- No genuineness
- Source parts from different manufacturers
- Open designs: people mix and match (typically screen and MCU)

Pre-built device

- Genuineness + easy to inspect
- Implant-proof: single secure chip (smartcard, Daric), or hard-to-break pairing between non-secure and secure chips
- Counter screen implants with split mnemonic generation

RESOURCES

My Devcon 8 talks



https://github.com/btchip/devcon8

Crossbar Dabao hackathon



https://crossbar-inc.com/pages/dabao-hackathon



THANK YOU!

NICOLAS BACCA (BTCHIP)
Chef, zknox